LEARN BEING FORWARD

Programming — TypeScript Programming

Generics

COMPLETE LEARNING GUIDE

What You'll Learn

Generics let a function, interface, or class be written once and work correctly with many different types, while still preserving full type safety — without generics, you'd either duplicate code for every type or fall back to `any` and lose type checking entirely. This sub-module builds generics from first principles: why they exist, generic functions and the type-parameter inference the compiler performs automatically at call sites, generic interfaces, type aliases, and classes, constraining a type parameter with extends so it must have certain properties, supplying default type parameters, the keyof and mapped-type machinery that powers advanced generic utilities, and the small set of built-in utility types (Partial, Pick, Omit, Record, ReturnType) that cover the overwhelming majority of real-world generic needs without writing custom type-level code.

1. Why Generics: The Problem They Solve

Without generics, a reusable function like 'return the first element of an array' either needs to be duplicated for every element type, or typed with `any`, which discards type safety for every caller. A generic type parameter (conventionally named T) acts as a placeholder that gets filled in with a real type at each call site, letting one function definition work correctly — and with full type checking — for arrays of numbers, strings, or any other type, all without writing it more than once.

TYPESCRIPT

```
// Without generics: unsafe, loses all type information
function firstUnsafe(arr: any[]): any {
  return arr[0];
}

// With generics: safe, preserves the actual element type
function first<T>(arr: T[]): T {
  return arr[0];
}

const n = first([1, 2, 3]);        // inferred as number
const s = first(["a", "b"]);       // inferred as string
```

2. Generic Functions and Type-Parameter Inference

A generic function declares its type parameters in angle brackets before the parameter list (function identity<T>(value: T): T), and in the vast majority of calls you never specify the type argument explicitly — the compiler infers T from the actual argument passed in, exactly as it infers ordinary variable types. Explicit type arguments (identity<string>("hi")) are only needed on the rare occasion the compiler can't infer enough information from the arguments alone, such as a generic function called with no arguments that determine T.

```typescript
function wrapInArray<T>(value: T): T[] {
  return [value];
}

const nums = wrapInArray(5);         // inferred: number[]
const strs = wrapInArray("hello");   // inferred: string[]
const explicit = wrapInArray<boolean>(true); // explicit type argument, rarely needed
```

3. Generic Interfaces and Type Aliases

Interfaces and type aliases can themselves be generic, parameterized the same way as functions, which is the standard way to describe a container or wrapper shape that holds a value of some other, caller-specified type — an API response envelope, a paginated list, a result-or-error wrapper. Supplying the type argument when using the generic interface (ApiResponse<User>) fills in every occurrence of T throughout that interface's definition with the concrete type.

```typescript
interface ApiResponse<T> {
  data: T;
  status: number;
  error?: string;
}

interface User {
  id: number;
  name: string;
}

const response: ApiResponse<User> = {
  data: { id: 1, name: "Zoe" },
  status: 200,
};
```

4. Generic Classes

A class can declare its own type parameters (class Box<T> {}) that apply across every member — fields, method parameters, and method return types — which is how type-safe, reusable container classes (a typed Stack, Queue, or cache) are built once and specialized per-instance at construction time via the type argument supplied where the class is instantiated.

```typescript
class Stack<T> {
  private items: T[] = [];

  push(item: T): void {
    this.items.push(item);
  }

  pop(): T | undefined {
    return this.items.pop();
  }

  get size(): number {
    return this.items.length;
  }
}

const numberStack = new Stack<number>();
numberStack.push(1);
numberStack.push(2);
console.log(numberStack.pop()); // 2, inferred as number | undefined
```

5. Constraining Type Parameters With extends

By default a generic type parameter could be anything, which means the compiler won't let you access any property on it — T extends SomeShape constrains T to only types that satisfy SomeShape, which both restricts what callers can pass in and, in exchange, lets the function body safely access any property that SomeShape guarantees. This is the standard way to write a generic function that still needs to read specific properties (like .length or .id) off its generic argument.

```typescript
interface HasLength {
  length: number;
}

function logLength<T extends HasLength>(value: T): T {
  console.log(value.length); // safe — T is guaranteed to have .length
  return value;
}

logLength("hello");       // ok — strings have .length
logLength([1, 2, 3]);     // ok — arrays have .length
// logLength(42);         // error: number doesn't have .length
```

6. Default Type Parameters

A generic type parameter can declare a default (interface Container<T = string>) that's used when the caller doesn't explicitly supply a type argument and the compiler can't infer one from context either — this keeps simple, common use cases free of boilerplate type arguments while still allowing full customization when needed.

```typescript
interface Container<T = string> {
  value: T;
}

const defaultContainer: Container = { value: "uses the default string type" };
const numberContainer: Container<number> = { value: 42 };
```

7. keyof, typeof, and Mapped Types

keyof T produces a union of all of T's property-name string literals, which combined with a generic function lets you write a fully type-safe 'get a property by name' helper where the key and the return type are both checked against the actual object type. typeof (in a type context, distinct from the runtime typeof operator) extracts the type of a variable or value. A mapped type ({ [K in keyof T]: ... }) transforms every property of an existing type uniformly — this is exactly the mechanism behind built-in utilities like Partial and Readonly under the hood.

```typescript
interface Person {
  name: string;
  age: number;
}

function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const person: Person = { name: "Ivan", age: 40 };
const age = getProperty(person, "age");   // inferred as number
// getProperty(person, "email");          // error: "email" is not a key of Person
type ReadonlyVersion<T> = { readonly [K in keyof T]: T[K] }; // a hand-rolled mapped type
```

8. Essential Built-In Generic Utility Types

TypeScript ships a set of generic utility types built from the same mapped-type and keyof machinery, covering the overwhelming majority of everyday generic needs without writing custom type-level code: Partial<T> makes every property optional (useful for an update/patch payload), Pick<T, K> selects a subset of named properties, Omit<T, K> excludes named properties, Record<K, V> builds an object type with keys of K and values of V, and ReturnType<F> extracts a function's return type for reuse elsewhere without having to repeat it manually.

• Partial<User> for a PATCH/update payload where every field is optional; Pick<User, 'id' | 'name'> for a trimmed-down projection; Omit<User, 'password'> to exclude a sensitive field.

```typescript
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
}

type UserUpdate = Partial<User>;                  // every field optional
type PublicUser = Omit<User, "password">;         // all fields except password
type UserSummary = Pick<User, "id" | "name">;     // only id and name
type UsersById = Record<number, User>;            // { [id: number]: User }
```

Common Pitfalls

• Writing a generic function but immediately casting its parameter to `any` inside the body, which defeats the entire purpose of making it generic.

• Supplying explicit type arguments everywhere out of habit, when the compiler would correctly infer them from the arguments passed in.

• Forgetting a constraint (T extends HasProperty) and then being unable to access any property on a generic value inside the function body.

• Confusing keyof (which produces a union of a type's property-name literals) with the runtime typeof operator, which inspects a value's runtime type.

• Reaching for a hand-written mapped type when a built-in utility (Partial, Pick, Omit, Record) already expresses exactly the same transformation.

• Using `any` instead of a proper generic parameter 'to make the error go away,' silently reintroducing the type-safety gap generics exist to close.

Key Takeaways

• Generics let one definition work correctly, with full type safety, across many different types — the alternative is duplicating code or using unsafe `any`.

• Type-parameter inference usually eliminates the need to write explicit type arguments at a generic function's call site.

• T extends SomeShape constrains a type parameter so the function body can safely access properties that SomeShape guarantees.

• keyof T produces a union of T's property names; combined with a generic, it builds fully type-checked 'get/set by key' helpers.

• Mapped types ({ [K in keyof T]: ... }) transform every property of a type uniformly — it's the mechanism behind Partial, Readonly, and similar utilities.

- Partial, Pick, Omit, and Record cover most everyday generic object-shape transformations without writing any custom type-level code.

Learn More

TypeScript Handbook — Generics
https://www.typescriptlang.org/docs/handbook/2/generics.html

TypeScript Handbook — Keyof Type Operator
https://www.typescriptlang.org/docs/handbook/2/keyof-types.html

TypeScript Handbook — Utility Types
https://www.typescriptlang.org/docs/handbook/utility-types.html